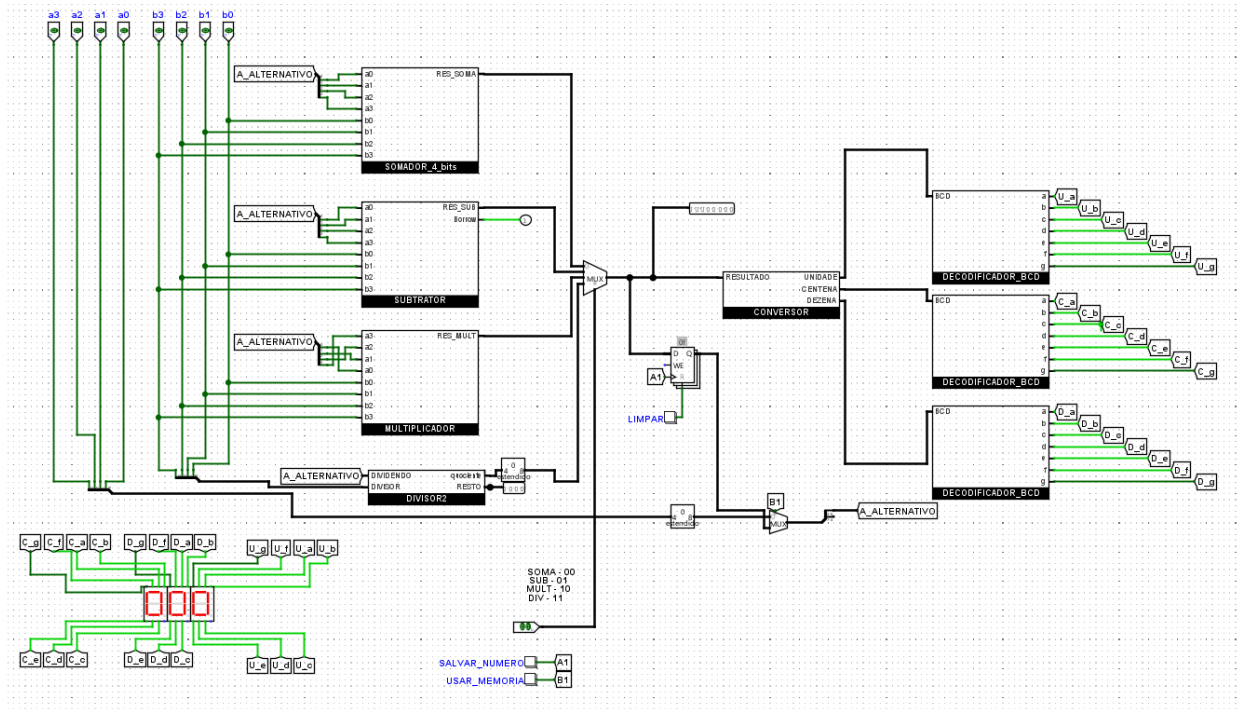


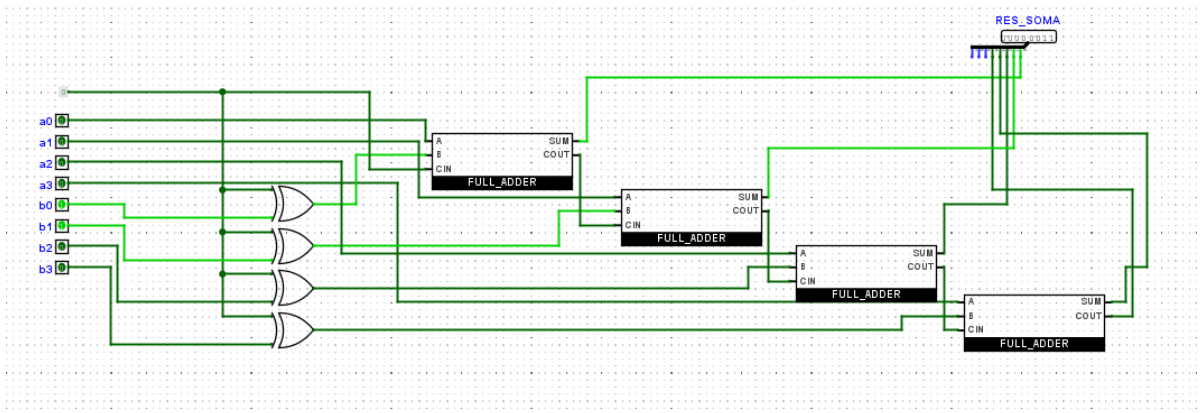
Report - 4-bit Calculator (Logisim)

(THIS REPORT WAS TRANSLATED USING AI AND MAY CONTAIN SOME ERRORS)



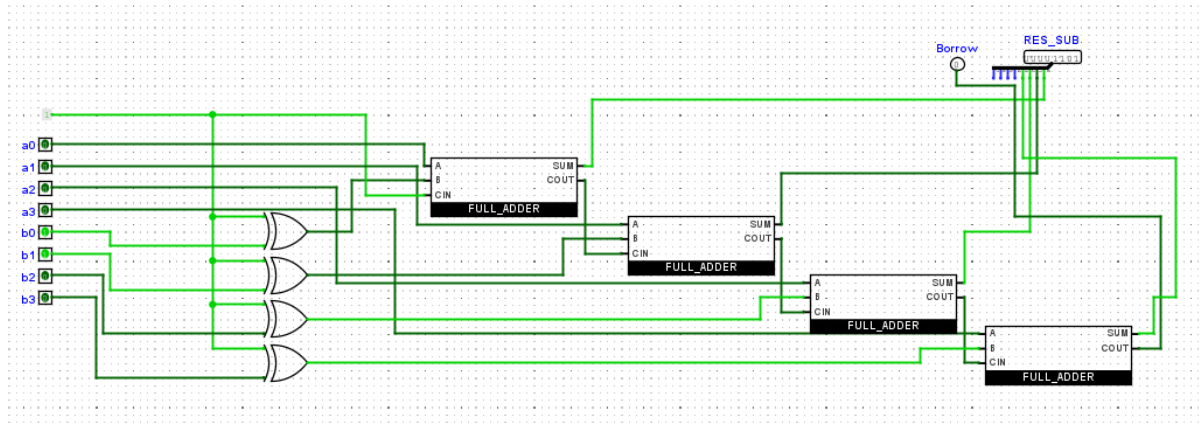
The calculator takes two 4-bit numbers (A and B), selects the operation with a 2-bit selector, and shows the result in decimal on three seven-segment displays. It also has a memory to save and reuse a value.

Addition



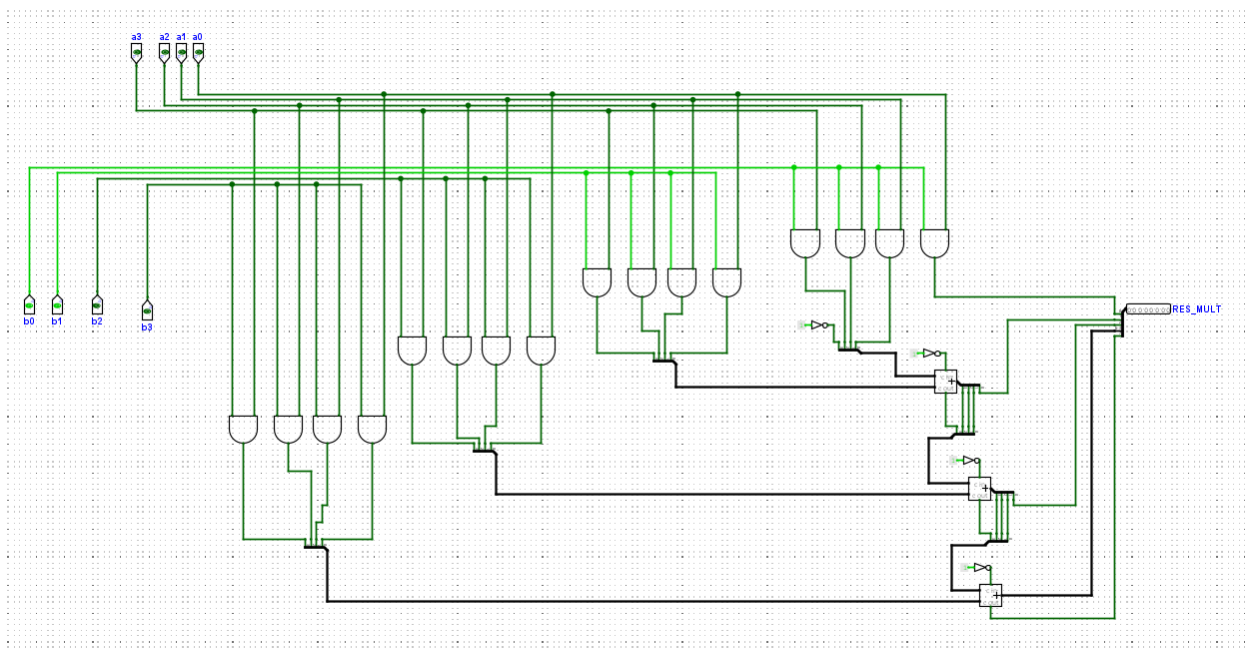
It uses 4 full adders (FULL_ADDER) connected in sequence, each one passing its carry to the next. Each adder computes $SUM = A \oplus B \oplus CIN$ and $COUT = (A.B) + (B.CIN) + (A.CIN)$.

Subtraction



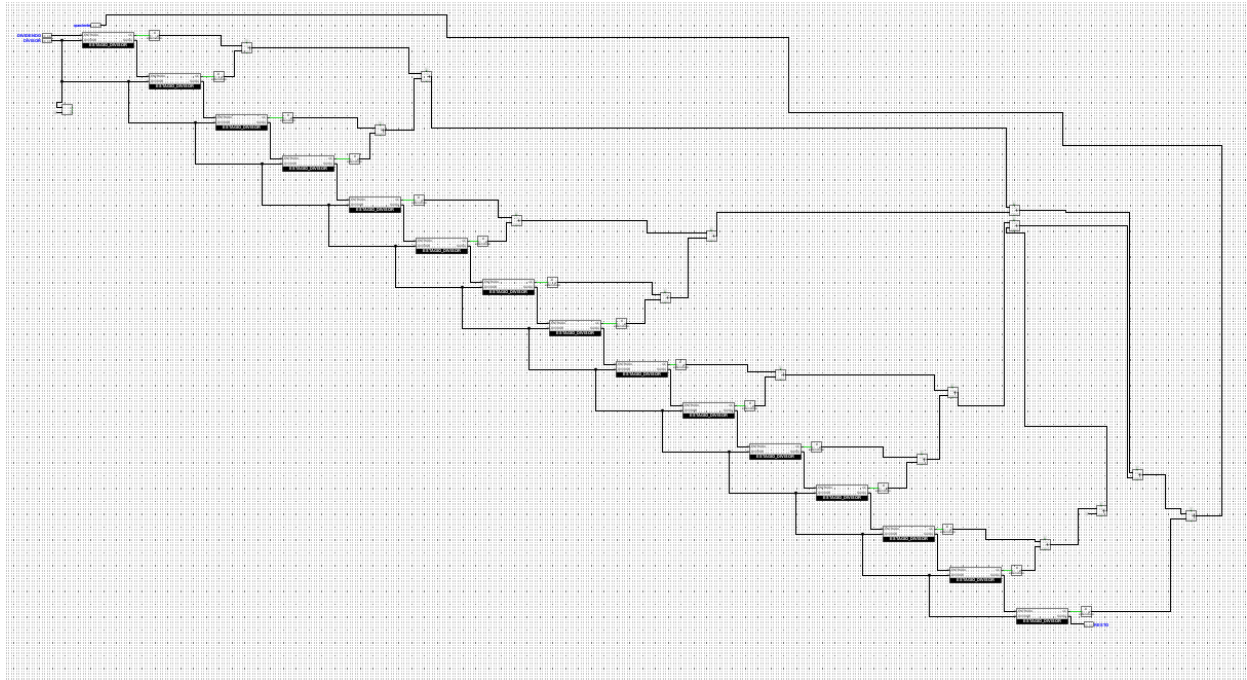
It uses the same chain of 4 adders as the addition circuit. The difference is that each bit of B passes through an XOR gate with a value of 1, inverting B, and the initial carry is also set to 1. This implements $A - B$ using the two's complement method ($A + \sim B + 1$).

Multiplication



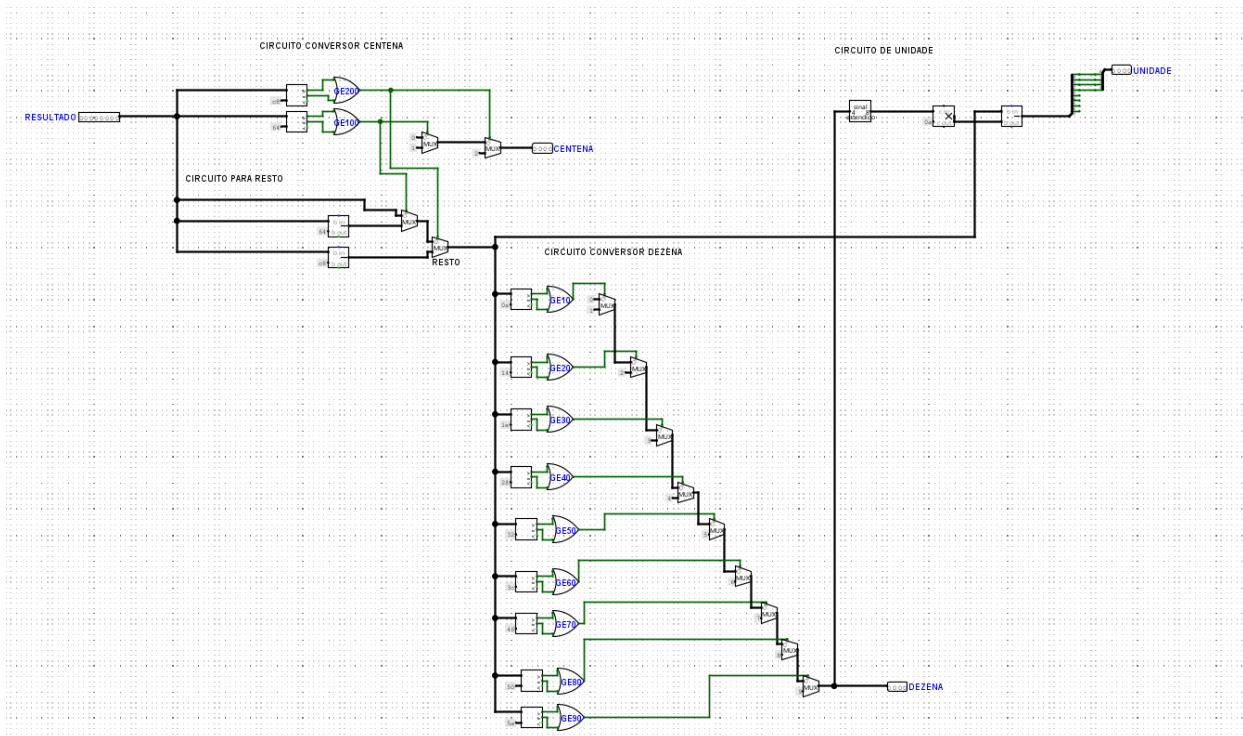
It uses an array multiplier: 16 AND gates generate all the partial products between the bits of A and B, and 3 adders sum these shifted rows to form the final 8-bit result.

Division



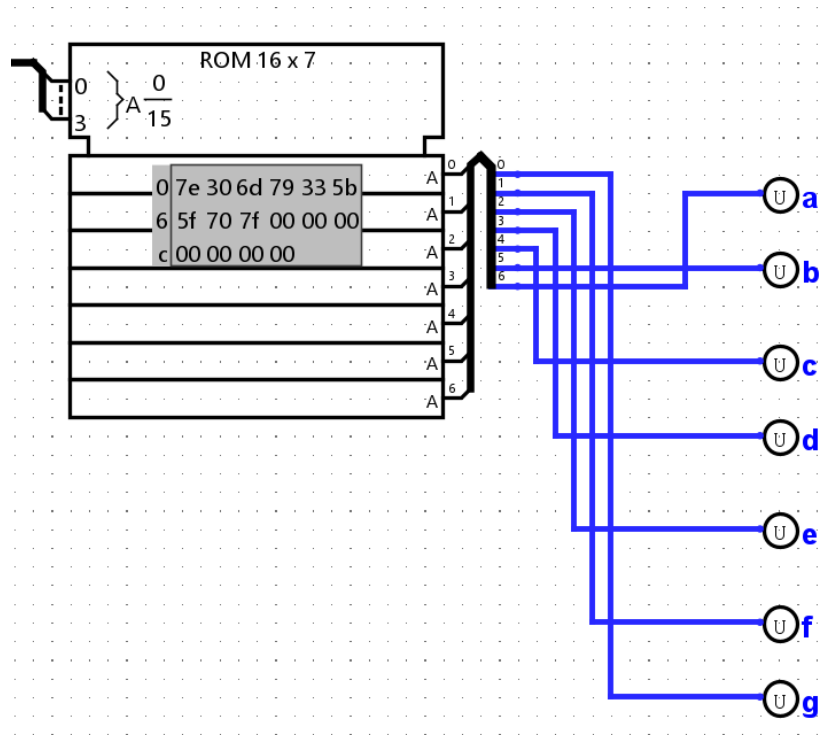
It uses the restoring division method, built from 15 repeated blocks (ESTAGIO_DIVISOR) connected in a chain. Each block compares the remainder with the divisor, performs the subtraction, and a multiplexer decides whether to keep the subtraction result (when the divisor fit) or undo it (restore) when it did not. This generates the quotient and remainder, bit by bit.

Conversion to decimal



Since results can reach up to 225, a converter extracts the three decimal digits (hundreds, tens, units) by comparing the value with 100 and 200 (for the hundreds digit) and then with multiples of 10 (for the tens digit), subtracting each time the value is greater than or equal to the tested multiple. What remains at the end is the units digit.

Seven-segment display



Each decimal digit (0 to 9) is converted into the correct segment pattern using a ROM memory: the digit is used as the address, and the ROM returns the seven-segment code. This same block is repeated 3 times, one for each display.

Operation selection

A multiplexer with a 2-bit selector chooses which result to display: 00 = addition, 01 = subtraction, 10 = multiplication, 11 = division. All four operations are calculated continuously; only the selected result is shown.

Memory

An 8-bit register stores a value; the result output feeds into the register's data input. Three buttons control this function:

- SALVAR_NUMERO (SAVE_NUMBER): writes the current value into the register.
- USAR_MEMORIA (USE_MEMORY): uses the stored value as an operand, in place of the normal input.
- LIMPAR (CLEAR): resets the register to zero.